Resume Pilot

Izabela Camaj

izabelacamaj@oakland.edu

Computer Science and Engineering Department

School of Engineering and Computer Science

CSI 5130: Artificial Intelligence

Dr. Tianle Ma

December 1, 2025

Abstract

Resume Pilot is a tool developed to help users improve their resumes. This tool prompts the users to attach their resume file and click a button to start the analysis process. In order to provide improvement suggestions and notes, the analysis process employs AI techniques, such as probabilistic classification, Markov Decision Process (MDP), Deep Q-Learning, Policy Gradient scoring, and a PyTorch self-attention module. It is a full-stack system, so the backend is built with FastAPI in Python and the frontend is built with CSS, JS, and HTML.

Introduction

The professor granted us freedom before taking on the development of this project, so I thought about addressing a thing that seems to be taking up most of my free time: internship applications. Now more than ever before, college students are expected to graduate with internship experience in the field that they are majoring in. The competition in the job market prompted this development, so students are encouraged to start applying for internships as early as freshman year. This is not necessarily a bad thing because real-world experience is very beneficial for professional development and determining whether you are sincerely interested in your major or a future job that students might be planning for. Though, students have to consider this additional aspect of their career development earlier than they would have before. The application process has also become more rigorous and I constantly find myself changing my resume. With this in mind, I was inspired to create Resume Pilot.

The primary users of this website include people that are completing professional job applications, such as college students seeking internships, adults looking for full-time jobs, employees looking for other positions at other companies, etc. Once the user has reached the main page of Resume Pilot, they will see two sections: Scan Attachments and Scan Results.

Under the “Scan Attachments" section, the user will be prompted to insert a pdf only file of their resume. Once applied, the user can click the “Analyze” button, which will prompt a return in the “Scan Results” section. The pdf file is parsed and analyzed automatically, before the system returns a personal information, predicted job field, skill recommendations, improvement actions, a score, and attention-based importance. This is possible with the application of AI concepts studied in class, such as probabilistic classification, Markov Decision Process (MDP), Deep Q-Learning, Policy Gradient scoring, and a PyTorch self-attention module. Probabilistic classification is utilized to provide a career prediction, MDP helps suggest resume improvements, Deep Q-Learning helps with skill recommendation, a policy gradient is used to provide a resume score, and Self-Attention in Pytorch is used to highlight important text. This is a full-stack application, so I have utilized Python, Javascript, CSS, and HTML for development.

Method

First, I set out realistic goals that I hoped to meet by utilizing concepts we have been learning in our Artificial Intelligence course. After this determination, I created a UI design as noted in Fig. 1.

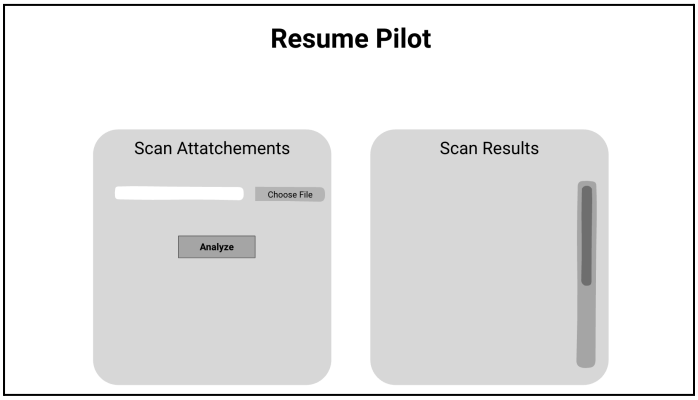

Fig. 1: UI Design

The project architecture includes a Frontend folder, a folder for AI models, a data folder, and the app.py file. The UI allows users to upload PDF only resumes files and displays all

analysis results. The Frontend files, such as HTML, CSS, and JS, send the PDF resume files to the backend with the /analyze endpoint. The user can then see results under the “Scan Results” from the AI models: field prediction, skills, score, improvements, and keywords.

When considering the backend, the app.py file is the central controller. So, it serves frontend files and shares the /analyze API endpoint. CORS middleware is also integrated in order for the browser to communicate with the backend. This is important because you will not be able to block the POST /analyze request. This file also processes the uploaded PDF documents and extracts text before integrating the AI models. Later, it adds the model outputs in a JSON response which will be sent to the frontend.

The AI models currently return career prediction, resume improvements, skill recommendation, resume score, highlights, and keywords. The probabilistic_model.py helps to predict the users’ career field by implementing TF-IDF and Logistic Regression. The skill_dqn_agent.py file creates recommendations that can be missing skills by referring to the learned skill maps. The pg_score_model.py file finalizes the resume quality score by adding section coverage and word count, which is specifically policy gradient-based. The resume_mdp_model.py file makes suggestions for actions, such as “add”, “improve”, “skip”, with an MDP. The resume_attention_module.py file utilizes PyTorch self-attention to highlight important words in the resume text. The train_models.py file is a training script that generates .pkl model files. These files are used at runtime and include field_model.pkl, resume_vectorizer.pkl, skill_map.pkl, and score_stats.pkl files. It builds the classifier, vectorizer, skill map, and scoring stats.

The data folder stores the training dataset, titled Resume.csv, and uploaded user resumes. Resume.csv is utilized to train the AI models. While sharing my progress, I noticed that files

were being shared under the uploaded_resumes folder. So, I implemented .gitignore in order to protect the virtual environment and uploaded resumes.

Experimental Results

I ran into many issues due to dependencies, so I installed.venv, or a virtual environment, in order to isolate FastAPI, PyTorch, pdfminer, joblib, etc. This helped my project run consistently across different machines. After running the code and following the browser link, the user is met with the main UI page. It successfully includes the title, welcome text, and two sections that allow users to submit resume PDF files and review scan results.

Initially, I was met with a few connection issues. I discovered that the returned data could not be trusted because the Resume dataset had not been connected. I discovered this when I clicked the "Analyze" button to upload the same file and received a different resume score. When showcasing this project to different friends and family members, everyone recommended an additional feature that would allow the user to also post specific job requirements and compare the resumes against that information. I attempted to add a text bow for job requirements in the UI, parse and extract keywords from it, compare resume keywords with job keywords, and add improvement suggestions accordingly. Though, I had trouble with incorrect keyword matching and handling large skill words. In the event that I had more time to add this, or had heard this recommendation earlier, I would incorporate this feature.

In the .venv, I learned that it is important to install necessary packages. FastAPI allowed me to create a clean backend. The browser cannot read or extract PDF files, but the PDFMiner could if it runs on a server. By installing FastAPI, the backend could allow file uploads. The /analyze endpoint took in PDF uploads and returned a JSON response. The additional CORS middleware allowed the browser to communicate with the backend.

At first, I planned to use Streamlit but it restricted UI customizations that I was hoping to apply due to my limited experience. I also hoped to apply PyResparser, but it caused many parsing issues, such as error handling and incorrect name extraction. This package also required outdated libraries, which led to installation failures. Because of this, I decided to utilize tools such as FastAPI, pdfminer.six, and custom regex extraction for the development of Resume Pilot.

Conclusion

Resume Pilot is a tool that combines AI learning methods in order to perform resume analysis. I have developed a backend with FastAPI, which integrates classification, scoring, skill recommendations, and keyword extraction. The frontend allows users to upload resumes and view their scan results. During the development process, I have improved accuracy and reliability by modifying the parsing process and the filtering process. The analysis process successfully implements AI techniques, such as probabilistic classification, Markov Decision Process (MDP), Deep Q-Learning, Policy Gradient scoring, and a PyTorch self-attention module.

References

Canva. (n.d.). Canva design templates. https://www.canva.com/

FastAPI Team. (n.d.). Handling file uploads. *FastAPI Documentation.* https://fastapi.tiangolo.com/tutorial/request-files/

GitHub. (n.d.). GitHub documentation and platform. https://github.com/

Indeed Editorial Team. (n.d.). Resume samples and writing guide. *Indeed.* https://www.indeed.com/career-advice/resume-samples

Mozilla Developer Network. (n.d.). Cross-Origin Resource Sharing (CORS). https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS

OpenAI. (n.d.). Vanilla policy gradient (VPG). *Spinning Up in Deep RL.* https://spinningup.openai.com/en/latest/algorithms/vpg.html

pdfminer.six Contributors. (n.d.). pdfminer.six documentation. https://pdfminersix.readthedocs.io/en/latest/

Purdue Online Writing Lab. (n.d.). Résumés and CVs. https://owl.purdue.edu/owl/job_search_writing/resumes_and_vitas/index.html

PyTorch Team. (n.d.). *torch.nn.MultiheadAttention.* PyTorch Documentation. https://pytorch.org/docs/stable/generated/torch.nn.MultiheadAttention.html

Scikit-learn Developers. (n.d.). Text feature extraction using TF-IDF. *Scikit-learn.* https://scikit-learn.org/stable/modules/feature_extraction.html#text-feature-extraction

Stack Overflow. (n.d.). Stack Overflow: Developer community Q&A. https://stackoverflow.com/

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press. https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf

Tiangolo, S. (n.d.). *FastAPI documentation.* https://fastapi.tiangolo.com/

Uvicorn Developers. (n.d.). Uvicorn: ASGI web server. https://www.uvicorn.org/